

# H.I.V.E. — Hardened Isolation Vector Engine

---

## **Complete Engineering Documentation & Source-of-Truth Blueprint Privacy-First Local AI Runtime Platform**

This document defines the complete engineering specification, architectural contracts, operational mechanics, runtime boundaries, security mandates, directory standards, streaming protocols, RAG specifications, and failure recovery mechanisms for the H.I.V.E. platform.

### **1. System Overview & Core Objectives**

- Fully local AI runtime with zero cloud inference dependencies.
- Server-Sent Events (SSE) streaming architecture.
- Autonomous tool-calling agentic middleware loop.
- Retrieval-Augmented Generation (RAG) support.
- Air-gapped privacy-first deployment capability.
- Hardware-aware VRAM-safe inference execution.

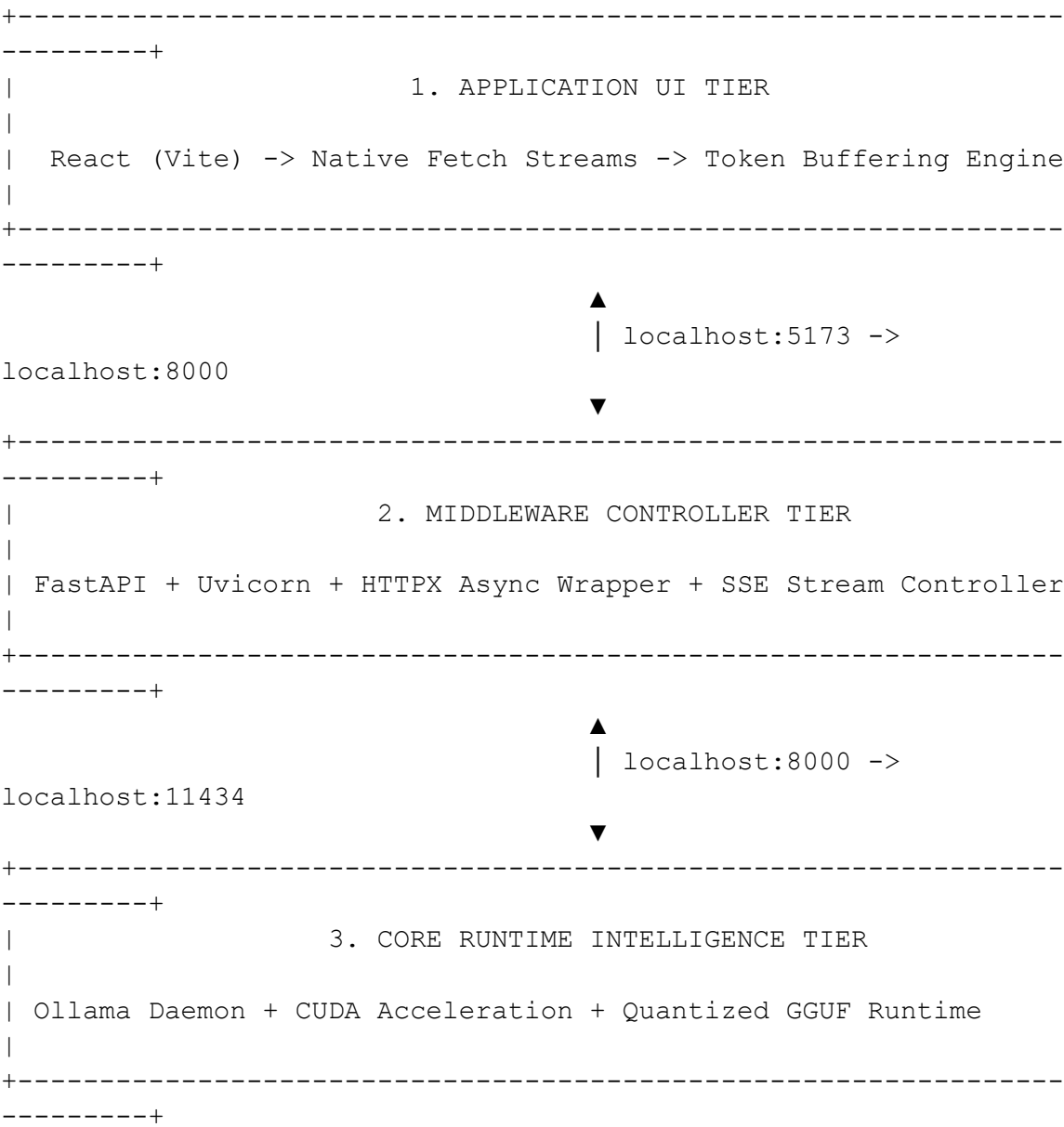
### **2. Developer Hardware Stack**

- CPU: Intel Core i7-13700HX (16 Cores).
- GPU: NVIDIA RTX 4060 Laptop GPU (8GB VRAM, CUDA-enabled).
- Secondary GPU: Intel UHD Graphics.
- RAM: 16GB System Memory.
- Storage: 1TB NVMe SSD.
- Operating System: Windows 11 Home Single Language.

### **3. Hardware Boundary Architecture**

- Windows reserves approximately 0.8GB–1.2GB VRAM for rendering overhead.
- Maximum stable inference budget: 6.5GB–7.0GB VRAM.
- Crossing VRAM boundaries causes Shared Memory overflow.
- PCIe fallback causes generation performance collapse.
- All production models must remain quantized and VRAM-safe.

4. Three-Tier Architecture



5. Unified JSON Data Contracts

Frontend Request Payload Example:

```
{
  "model": "llama3.1",
  "messages": [
    {
```

```

        "role": "system",
        "content": "You are H.I.V.E."
    },
    {
        "role": "user",
        "content": "Explain SSE streaming."
    }
],
"temperature": 0.4
}

```

### Server-Sent Events Streaming Payload Example:

```
data: {"token":"Hello","done":false}
```

```
data: {"status_update":"Searching live web...","done":false}
```

```
data: {"token":"","done":true}
```

## 6. Repository Directory Scaffolding

```

hive-root/
├── backend/
│   ├── app/
│   │   ├── api/           # chat routes, SSE endpoints
│   │   ├── core/         # config.py, security, slowapi
│   │   ├── services/     # ollama_client.py, rag_service.py
│   │   ├── models/       # Pydantic schemas
│   │   └── utils/        # helpers, parsers
│   ├── requirements.txt
│   ├── .env.example
│   └── main.py
├── frontend/
│   ├── src/
│   │   ├── components/   # ChatWindow, Sidebar
│   │   ├── hooks/        # useOllamaStream.js
│   │   ├── services/     # fetch wrappers
│   │   ├── store/        # state containers
│   │   └── App.jsx
│   └── vite.config.js

```

```

├── package.json
├── docs/
│   └── architecture/
├── vector_store/
└── .gitignore

```

## 7. Autonomous Agentic Execution Flow

1. Frontend submits structured conversation arrays.
2. FastAPI injects tool schemas into inference payload.
3. Ollama evaluates whether tool execution is required.
4. tool\_calls payloads are intercepted server-side.
5. DuckDuckGo scraper executes in isolated middleware.
6. Context results are appended as role='tool'.
7. Final augmented inference stream is emitted to client.

## 8. Edge-Case Error Handling Matrix

| Failure Scenario            | Required Recovery Behavior                                         |
|-----------------------------|--------------------------------------------------------------------|
| Ollama daemon offline       | Emit SSE error payload: {"error": "H.I.V.E. Core Engine offline."} |
| Context window overflow     | Implement FIFO message pruning strategy.                           |
| Frontend network disconnect | Terminate active stream gracefully and cleanup tasks.              |
| GPU memory exhaustion       | Reject model load and fallback to lightweight runtime model.       |
| Malformed JSON payload      | Reject request with structured Pydantic validation response.       |
| Scraper timeout             | Inject fallback response stating live data unavailable.            |

## 9. Hardened Security Architecture

- All secrets isolated inside .env configuration.
- Frontend forbidden from external network communication.
- Server-side validation mandatory for every request.
- Parameterized database queries only.
- Rate limiting enforced using slowapi.
- Future databases require Row-Level Security (RLS).
- Tool execution isolated from direct client access.

## 10. RAG Implementation Specifications

- Embedding Model: bge-small or all-minilm.
- Vector Database: ChromaDB or FAISS.
- Chunking Strategy: Recursive Character Splitting.
- Chunk Size: 500 characters.
- Chunk Overlap: 50 characters.
- Supported Documents: PDF, Markdown, DOCX, TXT, CSV.
- Embeddings generated locally through Ollama runtime.

## 11. Engineering Enforcement Standards

- All Python backends must remain asynchronous.
- All client communication must use SSE streaming.
- No external APIs directly accessible from frontend.
- All runtime configuration isolated via environment variables.
- All services modularized into independent layers.
- All future features must extend this blueprint.

## 12. Living Documentation Mandate

This documentation is considered a living engineering source-of-truth. As H.I.V.E. evolves, every new API schema, module structure, RAG pipeline, database entity, error state, and streaming contract must be appended to this document to prevent architectural drift and technical debt.